

⚡ ADVANCED COMPILER ENGINEERING

LLVM Kaleidoscope

Compiler Architecture & Implementation

A Comprehensive Deep-Dive into Lexical Analysis, Recursive Descent Parsing, AST
Design, Semantic Scoping, and LLVM IR Generation

📁 **Repository:** [llvm-kaleidoscope](#)

⚙️ **Backend:** LLVM IR Builder & Core API

💻 **Language:** C++14 / Modern OOP

🔗 **Topic:** Compiler Frontend & IR Lowering

Project Motivation & Objectives



What is Kaleidoscope?

A procedural toy language designed to demonstrate complete compiler frontend architecture using the LLVM infrastructure.

- All numerical data are 64-bit IEEE floating point (`double`)
- First-class function definitions with `def`
- C-style standard library binding with `extern`
- Pure expression-based syntax



Engineering Goals

Transition from language theory to practical, modular C++ implementation with LLVM toolchains:

- High-speed tokenization without external tools (Flex/Lex)
- Recursive Descent with Operator-Precedence parsing
- Object-Oriented Abstract Syntax Tree (AST)
- Direct translation to verified Static Single Assignment (SSA) LLVM IR

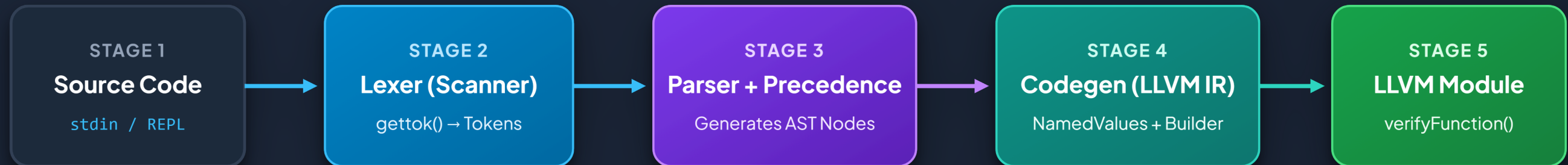


Codebase Structure

Refactored into clean, single-responsibility C++ translation units:

- `lexer/`: Token definitions and stream scanner
- `ast/`: 7 modular polymorphic AST class definitions
- `parser/`: Binary precedence & grammar engine
- `kaleidoscope/`: LLVM Context, Module, and Builder
- `logger/`: Error reporting and recovery

High-Level Compiler Pipeline



Interactive REPL Driver

The driver in `main.cpp` operates in a continuous loop, classifying user input into definitions, extern declarations, or immediate top-level expressions wrapped in anonymous functions.

Verifier & Integrity Guard

Every function generated is checked by LLVM's `verifyFunction` pass, catching type mismatches, invalid basic block terminations, or unassigned SSA registers.

Lexical Analysis (Lexer Design)



Scanner Architecture

The lexer is implemented as a stateful character-by-character scanner in `lexer.cpp` driven by `gettok()`.

- **Whitespace Filtration:** Automatically skips spaces, tabs, and linebreaks via `isspace()`.
- **Identifier & Keyword Buffer:** Accumulates alphanumeric characters into `IdentifierStr`. Returns `tok_def`, `tok_extern`, or `tok_identifier`.
- **Numeric Literal Processing:** Matches continuous digit sequences and decimal points into `NumVal` via `strtod()`.
- **Comment Stripping:** Scans from `#` to end-of-line and recursively requests next token.
- **Operator Passthrough:** Returns unknown characters (e.g. `+`, `*`, `()`) as their raw ASCII integer value `[0-255]`.

```
// Token Enumeration in lexer/token.h
enum Token {
    tok_eof = -1,
    // Commands
    tok_def = -2,
    tok_extern = -3,
    // Primary Entities
    tok_identifier = -4,
    tok_number = -5
};

// Scanning Logic in lexer/lexer.cpp
if (isalpha>LastChar)) {
    IdentifierStr = LastChar;
    while (isalnum>LastChar = getchar()))
        IdentifierStr += LastChar;

    if (IdentifierStr == "def") return tok_def;
    if (IdentifierStr == "extern") return tok_extern;
    return tok_identifier;
}
```

Parser Architecture & Operator Precedence



Operator Precedence Parsing

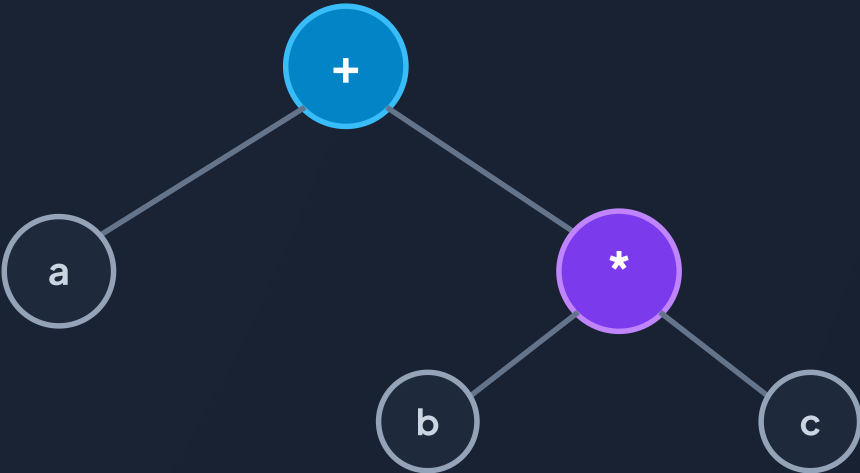
Kaleidoscope utilizes a hybrid **Recursive Descent Parser** for grammar rules combined with an **Operator-Precedence Climbing** algorithm for mathematical binary expressions.

Precedence Hierarchy Table		
< : Priority 10	+, - : Priority 20	*, / : Priority 40

- `ParsePrimary()` routes tokens to numbers, parenthesized expressions, or variables/function calls.
- `ParseBinOpRHS()` dynamically groups subtrees based on comparative token precedence.



Parsing: `a + b * c` Expression Tree



Operator precedence ensures `*` binds tighter to `b` and `c`, forming the subtree before attaching to `+`.

Abstract Syntax Tree (AST) Class Hierarchy

1 NumberExprAST

Captures numeric constants like `42.0`.

```
double Val;  
codegen();
```

2 VariableExprAST

References bound arguments (e.g. `x`).

```
string Name;  
codegen();
```

3 BinaryExprAST

Represents operations: `LHS op RHS`.

```
char Op;  
unique_ptr LHS, RHS;
```

4 CallExprAST

Represents invocations: `sin(x, y)`.

```
string Callee;  
vector Args;
```

5 PrototypeAST

Captures function metadata: signature name and parameter identifiers list. Emits `llvm::Function*` declarations.

6 FunctionAST

Combines a `PrototypeAST` with a body `ExprAST`. Manages the function entry basic block and populates argument symbol tables.

Semantic Analysis & Scope Resolution



Symbol Table Management

Managed globally via `NamedValues` map:

```
std::map<string, llvm::Value*>  
NamedValues;
```

- Scoped per function definition
- Cleared upon each new function body compilation
- Populated with named function formal parameters



Type Checking & Domain

Kaleidoscope enforces a unified floating-point model:

- All numerical expressions resolve to `Type::getDoubleTy(TheContext)`.
- Boolean comparisons (`<`) evaluate to 1-bit int and are immediately cast to `double` via `CreateUIToFP`.
- Arity verification: Function call argument count is strictly verified against prototype signature.



Error Recovery Strategy

Robust logging prevents compiler crashes:

- `LogError`: Emits syntax errors to `stderr` and returns `nullptr`.
- `LogErrorV`: Catches unknown variables or incorrect argument counts during codegen.
- If codegen fails, `TheFunction->eraseFromParent()` rolls back partial LLVM IR to maintain module integrity.

LLVM IR Code Generation Strategy

LLVM Core Infrastructure

- **LLVMContext (TheContext):** Core environment owning type tables, constant pools, and memory arenas.
- **IRBuilder (Builder):** High-level cursor that inserts instructions into basic blocks and performs constant folding.
- **Module (TheModule):** Top-level container holding compiled functions, symbol tables, and external declarations.
- **SSA Form Guarantee:** Generates instructions directly in Static Single Assignment form without mutable stack allocations.

```
// BinaryExprAST::codegen() in ast/BinaryExprAST.cpp
llvm::Value *BinaryExprAST::codegen() {
    llvm::Value *L = LHS->codegen();
    llvm::Value *R = RHS->codegen();
    if (!L || !R) return nullptr;

    switch (Op) {
        case '+': return Builder.CreateFAdd(L, R, "addtmp");
        case '-': return Builder.CreateFSub(L, R, "subtmp");
        case '*': return Builder.CreateFMul(L, R, "multmp");
        case '<':
            L = Builder.CreateFCmpULT(L, R, "cmptmp");
            return Builder.CreateUIToFP(L,
                llvm::Type::getDoubleTy(TheContext), "booltmp");
    }
}
```


Execution Walkthrough: Compiling a Function

Target Source Input: `def foo(x y) x + y * 4.0;`

1. Tokens Generated

```
[tok_def] "def"  
[tok_identifier] "foo"  
['(']  
[tok_identifier] "x"  
[tok_identifier] "y"  
[')']  
[tok_identifier] "x"  
['+']  
[tok_identifier] "y"  
['*']  
[tok_number] 4.0  
[';']
```

2. AST Built

```
FunctionAST(  
  Prototype: foo(x, y),  
  Body: BinaryExprAST(  
    Op: '+',  
    LHS: Variable("x"),  
    RHS: BinaryExprAST(  
      Op: '*',  
      LHS: Variable("y"),  
      RHS: Number(4.0)  
    )  
  )  
)
```

3. Emitted LLVMIR

```
define double @foo(double %x, double %y) {  
  entry:  
    %multmp = fmul double %y, 4.000000e+00  
    %addtmp = fadd double %x, %multmp  
    ret double %addtmp  
}
```

Functionality Audit & Gap Analysis

Compiler Feature	Implementation Status	Notes & Architectural Details
Lexical Analysis & Tokens	✓ Completed	Numbers, identifiers, def, extern, comments (#), ASCII operators.
Operator Precedence Parser	✓ Completed	Recursive descent climbing for <, +, -, *, calls, parens.
LLVM IR Code Generation	✓ Completed	Generates valid LLVM SSA instructions with verifyFunction integrity checks.
JIT Runtime Execution Engine	✗ Missing	Top-level expressions emit IR as __anon_expr but are not executed in memory.
Control Flow (if/then/else, loops)	⌚ Planned	No conditional basic block branching (Tutorial Chapter 5).
Mutable Variables & Memory	⌚ Planned	No var keyword or memory stack allocations (alloca) (Tutorial Chapter 7).
PassManager Optimizations	⌚ Planned	No constant folding, dead code elimination, or reassociation passes attached.

Future Work Plan & Implementation Milestones

P1 Phase 1: JIT & Optimization

Milestone 1 • Chapter 4

- Integrate **LLVM OrcJIT / LLJIT** to evaluate anonymous expressions immediately.
- Attach **FunctionPassManager** with Instruction Combining, GVN, and Reassociate passes.
- Enable live REPL evaluation.

P2 Phase 2: Control Flow & State

Milestone 2 • Chapters 5–7

- Implement **IfExprAST** with conditional branch phi-nodes.
- Add **ForExprAST** loop construct.
- Add mutable local variables with **stack a11oca** and **LLVM mem2reg** pass.

P3 Phase 3: Native Target Emission

Milestone 3 • Chapters 8–9

- Configure target machine triple and data layouts for cross-compilation.
- Emit native object code (**.o / .obj**) and link to standalone executables.
- Add **DWARF / CodeView** debug symbols.

 SUMMARY & WRAP-UP

Thank You! Questions & Discussion

Key Project Takeaways:

- Clear separation of concerns between Tokenizer, Grammar Parser, and LLVM Backend.
- Efficient AST architecture with clean polymorphic C++ memory management (`std::unique_ptr`).
- Demonstrates direct code generation to industrial-strength, verified LLVM Intermediate Representation.

 **Reference:** llvm.org/docs/tutorial

 **Repository:** [llvm-kaleidoscope](https://github.com/llvm-kaleidoscope)